Projeto 01 - Implementação da Cifra de Vigenère

João Gabriel Machado Ferreira
Departamento de Ciência da Computação
Matricula: 232003607

Arthur Luiz Lima De Araújo
Departamento de Ciência da Computação
Matricula: 232000472

Warley De Morais Borges
Departamento de Ciência da Computação
Matricula: 211045356

Ycaro Rodrigo Rocha Santana
Departamento de Ciência da Computação
Matricula: 222034397

***Abstract*—Este projeto faz a implementação da cifra de Vigenère e um ataque via análise de frequência com testes de Kasiski e Qui-Quadrado. Testes em português e inglês comprovam a vulnerabilidade da cifra em mensagens extensas, ressaltando a relevância da estatística na segurança computacional.**

***Index Terms*—Criptografia, Vigenère, Python, Segurança Computacional, Kasiski, Análise de Frequência.**

I. Introdução

A crescente demanda por comunicação digital segura torna essencial o estudo de mecanismos criptográficos que garantam confidencialidade, integridade e autenticidade dos dados trafegados em redes de computadores. Neste contexto, a cifra de Vigenère, um dos primeiros exemplos de cifra de fluxo com chave repetitiva, oferece uma base histórica e didática para compreender conceitos fundamentais como o uso de keystream, a importância do tamanho da chave e a vulnerabilidade a ataques de análise de frequência. Embora atualmente considerada frágil, seu estudo permite assimilar princípios que serão aprofundados em algoritmos simétricos modernos (AES, ChaCha20) e no projeto de protocolos de segurança (TLS, IPsec). O presente trabalho explora a implementação completa da cifra de Vigenère — cifragem e decifragem — e, em seguida, desenvolve um ataque de recuperação de chave por análise de frequência sobre duas mensagens cifradas (português e inglês), demonstrando na prática como a repetição da chave e a estrutura do idioma permitem quebrar a cifra.

Estrutura do relatório – O documento está organizado nas seguintes seções:

1) **Seção II** – Fundamentação Teórica.
2) **Seção III** – Ambiente Experimental e Análise de Resultados.
3) **Seção IV** – Conclusões.
4) **Seção V** – Bibliografia

II. Fundamentação Teórica

A. Cifra de Vigenère: Criptografia polialfabética

A cifra de Vigenère é uma cifra de fluxo polialfabética que generaliza a cifra de deslocamento (*shift cipher*). Enquanto esta última aplica o mesmo deslocamento a todos os caracteres da mensagem, a cifra de Vigenère utiliza uma chave de comprimento variável L que se repete seus caracteres ciclicamente até alcançar o tamanho do texto plano alvo, produzindo deslocamentos distintos para cada caractere ao longo do texto.

B. Formulação matemática

Seja o alfabeto mapeado a números inteiros $0, 1, \ldots, 25$. Uma chave é definida como $K = k_1 k_2 \ldots k_L$, com cada $k_i \in \{0, \ldots, 25\}$. Para uma mensagem $m = m_1 m_2 \ldots m_t$, o criptograma $c = c_1 c_2 \ldots c_t$ é gerado por:

$$c_i = \left(m_i + k_{((i-1) \bmod L)+1}\right) \bmod 26.$$

O deslocamento na posição i é, portanto, $k_{((i-1) \bmod L)+1}$. Como a chave se repete a cada L caracteres, as mesmas posições da chave são aplicadas aos caracteres do texto claro cujos índices são congruentes módulo L.

1) Exemplo prático: Considere a chave `"cafe"`: mapeando cada letra para seu valor numérico na cifra A0Z25 (A=0, Z=25), temos $k_1 = 2$ (c), $k_2 = 0$ (a), $k_3 = 5$ (f), $k_4 = 4$ (e). Aplicando-a à mensagem `"tellhimaboutme"`, obtém-se o processo mostrado na Tabela I.

TABLE I
Cifragem de `"tellhimaboutme"` com chave `"cafe"`

i	Texto claro	Chave	Desloc.	Cifrado
1	t (19)	c (2)	2	v
2	e (4)	a (0)	0	e
3	l (11)	f (5)	5	q
4	l (11)	e (4)	4	p
5	h (7)	c (2)	2	j
6	i (8)	a (0)	0	i
7	m (12)	f (5)	5	r
8	a (0)	e (4)	4	e
9	b (1)	c (2)	2	d
10	o (14)	a (0)	0	o
11	u (20)	f (5)	5	z
12	t (19)	e (4)	4	x
13	m (12)	c (2)	2	o
14	e (4)	a (0)	0	e

O criptograma resultante é `veqpjiredozxoe`.

C. Implicação para a segurança

A periodicidade da chave introduz uma vulnerabilidade: ao extrair do criptograma as subsequências formadas pelos caracteres nas posições congruentes a um mesmo índice módulo

L, cada uma dessas subsequências, das quais chamamos "streams", foi cifrada com uma cifra de deslocamento constante. Ou seja, para uma chave de tamanho L, a cada L caracteres do texto cifrado, temos caracteres cifrados pelo mesmo caractere da chave.

D. Análise de Frequência (Qui-Quadrado)

A análise de frequência baseada no teste de Qui-Quadrado (χ^2) é uma técnica estatística utilizada para inferir o deslocamento de uma cifra de substituição monoalfabética quando o alfabeto original é conhecido, mas a chave é oculta. Diferente da análise de frequência simples, que busca apenas a letra mais comum (como o 'E' no inglês ou o 'A' no português), o Qui-Quadrado avalia a distribuição de todo o conjunto de caracteres simultaneamente. Matematicamente, para uma sequência de caracteres de tamanho N, a estatística de aderência é definida pela fórmula:

$$\chi^2 = \sum_{i=a}^{z} \frac{(O_i - E_i)^2}{E_i}$$

Onde:

- O_i representa a frequência observada (contagem ou percentual) de cada letra no texto cifrado (ou dentro de uma "stream" de caracteres que assumimos terem sido deslocados pelo mesmo caractere da chave).
- E_i representa a frequência esperada daquela mesma letra na língua natural do texto plano.

O princípio teórico baseia-se na minimização do erro: quanto menor o valor de χ^2, maior é a correlação entre a amostra observada e a distribuição teórica do idioma. Na Cifra de Vigenère, após a decomposição do criptograma em L componentes (onde L é o período da chave), cada componente comporta-se como uma cifra de deslocamento fixa. Ao testar estatisticamente todos os 26 deslocamentos possíveis para cada componente, o valor de χ^2 que se aproxima de zero indica, com alta probabilidade, o caractere correto da chave para aquela posição específica.

E. Índice de Coincidência (IC)

O índice de coincidência mede a probabilidade de duas letras escolhidas aleatoriamente de um texto serem iguais. Para um texto em língua natural (inglês ou português), o IC típico situa-se entre 0,065 e 0,068; para uma sequência uniformemente aleatória, $IC \approx 1/26 \approx 0,0385$. Dado um criptograma, calcula-se o IC médio de suas L subsequências (formadas pelos caracteres nas posições congruentes a $i \bmod L$). O valor de L que maximiza esse IC médio é o comprimento mais provável da chave.

Observation 1. O teste do qui-quadrado aplicado no projeto não corresponde a uso do Índice de Coincidência, apesar de suas semelhanças. O qui-quadrado é aplicado dentro do ciclo que percorre os 26 possíveis deslocamentos, comparando a distribuição observada na stream com a distribuição esperada da língua (deslocada ciclicamente). O deslocamento que minimiza o valor do qui-quadrado é selecionado como mais provável.

Observation 2. É importante ressaltar que na implementação do código para este projeto, não foram utilizados os conceitos do Índice de Coincidência, mas que ele também consta como vulnerabilidade crucial da cifra de Vigenère, e portanto esta seção foi adicionada para identificar o método como uma abordagem alternativa para a quebra da cifra.

F. Exame de Kasiski

"Na criptoanálise, o método Kasiski (também conhecido como teste de Kasiski) é um método de ataque a cifras de substituição polialfabética, como a cifra de Vigenère. Foi publicado pela primeira vez por Friedrich Kasiski em 1863, mas parece ter sido descoberto de maneira independente por Charles Babbage já em 1846." [4] Consiste na análise de padrões repetidos de 2 ou 3 letras em um certo texto e análise das distâncias entre estes termos repetidos. No caso da análise de texto cifrado por cifra de Vigenère, os valores das distâncias entre termos repetidos e os divisores destes valores são possíveis candidatos a serem o tamanho da chave ou múltiplos do tamanho da chave que criptografou a mensagem. Este teste se baseia no conceito de que texto simples na linguagem comum tende a se repetir.

G. Recuperação da Chave por Análise de Frequência

Uma vez conhecido um valor candidato para L, cada subsequência é tratada como uma cifra de deslocamento. Para cada deslocamento $d \in [0, 25]$, decifra-se a subsequência e calcula-se o qui-quadrado entre a distribuição observada das letras e a distribuição esperada do idioma:

$$\chi^2 = \sum_{i=0}^{25} \frac{(O_i - E_i)^2}{E_i},$$

onde O_i é a contagem da i-ésima letra na subsequência decifrada e E_i é a contagem esperada (frequência teórica vezes o número total de letras). O deslocamento que minimiza χ^2 é escolhido como o caractere da chave naquela posição.

H. Tratamento de Espaços e Acentos

Para simplificar a análise de frequência, o código remove todos os espaços e caracteres não alfabéticos antes do processamento criptográfico, aplica a biblioteca `unidecode` para converter caracteres acentuados (ex.: "ç", "á") em seus equivalentes ASCII, e restaura os espaços após a cifração/decifração. Essa abordagem mantém a legibilidade do texto final sem interferir na contagem de letras para o ataque.

III. Ambiente experimental e Análise de resultados

A. Descrição do Cenário

O experimento foi conduzido em um ambiente de desenvolvimento local, focado na simulação de processos de criptografia simétrica e análise estatística de dados. A configuração do ambiente foi projetada para garantir a reprodutibilidade dos testes de cifração e a eficácia do algoritmo de ataque por frequência.

1) Hardware e Software: A execução dos algoritmos e o processamento das cadeias de caracteres foram realizados sob as seguintes especificações:

- **Processamento e Memória:** O ambiente de execução baseou-se em uma arquitetura x86_64, utilizando processamento multinúcleo para suportar as operações de busca exaustiva e cálculos estatísticos simultâneos.
- **Linguagem de Programação:** Foi utilizado o **Python 3.12.10**, selecionado por sua robustez no tratamento de *strings* e pela eficiência de suas estruturas de dados nativas (dicionários e iteradores).
- **Bibliotecas Adicionais:** Utilizou-se a biblioteca `unidecode` para o pré processamento de texto plano, garantindo a normalização de caracteres ASCII e a remoção de diacríticos que poderiam enviesar a análise de frequência.

B. Implementação

1) Implementação do Cifrador e Decifrador (Parte I): A primeira parte do código implementa a cifra de Vigenère clássica (ciphertext e chave compostas por caracteres alfabéticos), permitindo cifrar e decifrar mensagens textuais com uma senha. A seguir, cada atividade é descrita em detalhes, juntamente com os resultados observados.

a) Inicialização da classe e mapeamentos: No método `__init__` da classe `Vigenere`, são criados dois dicionários:

- `dicionarioA1Z26`: mapeia cada letra minúscula de "a" a "z" para os números de 1 a 26 (A1Z26).
- `dicionarioA1Z26_invertido`: mapeia os números de 1 a 26 de volta para as respectivas letras.

Observation 3. Para a facilitar a implementação do deslocamento, estamos utilizando A1Z26 ao invés do A0Z25 descrito anteriormente neste relatório.

Resultado: Esses dicionários permitem a conversão bidirecional entre letras e números, necessária para aplicar os deslocamentos aritméticos da cifra.

b) Método `conversor_A1Z26`: Recebe uma string (senha ou chave estendida) e converte cada caractere no número correspondente (1–26). Se a string tiver comprimento 1, retorna apenas o número; caso contrário, retorna uma lista de números.

Resultado: Gera uma representação numérica da chave, usada para determinar o deslocamento em cada posição.

c) Método `cifrador`: O processo de cifragem segue os seguintes passos:

1) **Converte mensagem para minúsculas** e armazena a versão original (com espaços).
   Resultado: Uniformidade (apenas letras minúsculas).
2) **Remove espaços** da mensagem.
   Resultado: Sequência contínua de letras.
3) **Aplica `unidecode`** para remover acentos.
   Resultado: Normalização para o alfabeto de 26 letras.
4) **Calcula o comprimento** da mensagem tratada.
   Resultado: Número de caracteres a cifrar.
5) **Estende a senha ciclicamente** até o tamanho da mensagem.
   Resultado: Chave com o mesmo comprimento da mensagem tratada.
6) **Converte a chave estendida para números** (1–26).
   Resultado: Lista de deslocamentos.
7) **Calcula o deslocamento** para cada caractere: $(\text{chave} + \text{letra} - 1) \mod 26$.
   Resultado: Letra cifrada para cada posição.
8) **Concatena as letras cifradas** em uma única string.
   Resultado: Criptograma contínuo (sem espaços).
9) **Restaura os espaços** originais usando a versão armazenada.

Resultado final do cifrador: Uma string contendo o texto cifrado, com letras minúsculas, sem acentos e com os mesmos espaços da mensagem original.

d) Exemplo: Para a senha `"senha"` e a mensagem `"Cuidado com o homem que fala com as mãos"`, o cifrador produz o seguinte criptograma:

```
uyvkavs pvm g lbtee uhl fspn joe ef tagw
```

e) Método `decifrador`: O decifrador executa passos análogos, mas com a operação inversa (subtração).

1) **Converte o criptograma para minúsculas** e armazena a versão original (com espaços).
   Resultado: Uniformização do texto.
2) **Remove espaços e acentos** (aplica `unidecode`).
   Resultado: Sequência contínua de letras cifradas.
3) **Estende a senha ciclicamente** do mesmo modo que no cifrador.
   Resultado: Chave com o mesmo comprimento do criptograma tratado.
4) **Converte a chave estendida para números** (1–26).
   Resultado: Lista de deslocamentos.
5) **Calcula o deslocamento inverso** para cada caractere: $(\text{letra_cifrada} - \text{deslocamento} + 1) \mod 26$.
   Resultado: Letra do texto claro recuperada.
6) **Restaura os espaços** originais usando a versão armazenada.

Resultado final do decifrador: A mensagem original em minúsculas, sem acentos. Para o exemplo anterior, a saída é:

```
cuidado com o homem que fala com as maos
```

(Observação: "mãos" $\rightarrow$ "maos" devido à remoção de acento.)

Os resultados obtidos são exatamente os esperados, comprovando que o par cifrador/decifrador é consistente.

f) Observações sobre o comportamento:

- **Acentuação:** A biblioteca `unidecode` converte caracteres acentuados em seus equivalentes ASCII (ex.: "ã" $\rightarrow$ "a", "é" $\rightarrow$ "e", "ç" $\rightarrow$ "c"), mantendo o alfabeto com exatamente 26 letras.

- **Espaços:** São completamente ignorados no cálculo dos deslocamentos (não avançam o índice da chave), mas são reinseridos na saída.
- **Maiúsculas/minúsculas:** Tanto a mensagem quanto o criptograma são convertidos para minúsculas; a cifra não preserva a capitalização original.
- **Outros caracteres:** O código atual não os trata; eles seriam removidos ou ignorados.

2) Ataque de recuperação de senha por análise de frequência (Parte II): Tomando como base os conceitos teóricos apresentados no vídeo *Cryptography - Breaking the Vigenere Cipher* [5] providenciado pelo roteiro de projeto, implementamos um atacante que visa descobrir a chave secreta que criptografou uma mensagem, tomando como entrada apenas o ciphertext. A segunda parte do código implementa um ataque à cifra de Vigenère baseado em dois passos principais:

1) Estimativa do comprimento da chave através do teste de Kasiski.
2) Recuperação da chave propriamente dita por análise de frequência de cada coluna (stream) do criptograma, utilizando o teste do qui-quadrado para comparar as distribuições observadas com as frequências esperadas da língua (português ou inglês).

A classe `Atacante` contém todos os métodos necessários, e o método principal é `decifraMeOuTeDevoro(ciphertext1,ciphertext2)` que processa duas mensagens cifradas (uma em português, outra em inglês) e retorna uma tupla de listas de chaves candidatas para cada uma das linguagens.

a) Estimativa do comprimento da chave – método `kasiski(ciphertext)`: **Objetivo:** Encontrar possíveis comprimentos da chave a partir de repetições no criptograma. Aplicando o Exame de Kasiski descrito anteriormente neste relatório.

Procedimento:

- Remove espaços e converte tudo para minúsculas.
- Identifica todos os trigramas (substrings de 3 caracteres) e regista as posições onde cada um ocorre.
- Filtra apenas os trigramas que aparecem pelo menos duas vezes.
- Se não existirem trigramas repetidos, repete o processo com **bigramas** (2 caracteres) (menos eficiente em encontrar o tamanho correto da chave).
- Para cada trigrama (ou bigrama) repetido, calcula as distâncias entre ocorrências consecutivas (caso haja ao menos 2 ocorrências do trecho).
- Para cada distância, determina todos os seus divisores (maiores ou iguais a 2).
- Conta a frequência com que cada divisor aparece (através do dicionário `distanciasPossiveis`).
- Ordena os divisores por ordem decrescente de frequência e devolve a lista ordenada.

Resultado: Uma lista de inteiros (`list[int]`) – Contendo os comprimentos de chave mais prováveis ordenada do mais provável até o menos provável. Se não houver repetições, o método retorna uma lista vazia.

b) Criação dos streams – método `criarStreams(ciphertext, m)`: **Objetivo:** Para um dado comprimento de chave m, separar o criptograma em m colunas (streams) que correspondem aos caracteres deslocados pelo mesmo caractere da chave (consequentemente pelo mesmo valor).

Procedimento:

- Remove espaços e converte para minúsculas.
- Para cada i de 0 a $m-1$, constrói uma lista com os caracteres nas posições $i, i+m, i+2m, \ldots$ do criptograma.
- Retorna uma lista de m listas de caracteres (cada lista é uma stream).

c) Cálculo das frequências – método `frequencia(stream)`: **Objetivo:** Contar a ocorrência de cada letra num stream.

Procedimento:

- Inicializa um dicionário com todas as letras de 'a' a 'z' com valor zero.
- Percorre o stream e incrementa o contador da letra correspondente.
- Retorna o dicionário com as contagens absolutas.

Observation 4. Dado que as frequências gerais de letras por língua providenciadas pelo roteiro de projeto como dados a serem utilizados como comparação neste projeto estão no formato de porcentagens, o dicionário gerado por este método é futuramente convertido de contagens de valores absolutos para porcentagens visando ser compatível e comparável com os dados axiomáticos providenciados.

d) Mapeamento do deslocamento – método `mapeamento(streamFrequency, lingua)`: **Objetivo:** Determinar qual deslocamento (0 a 25) melhor ajusta a distribuição de frequências do stream às frequências esperadas da língua (português ou inglês) providenciadas pelo roteiro de projeto [2], usando o teste do qui-quadrado. Este método enfim corresponde à análise de frequência necessária para a quebra da cifra.

Procedimento:

- Converte as contagens absolutas em **percentagens** (frequências relativas).
- Para cada possível deslocamento d de 0 a 25:
  - Compara a distribuição observada do stream com a distribuição esperada da língua **deslocada** (ao contrário, porque estamos a "reverter" o deslocamento da cifra).
  - Calcula o valor do qui-quadrado (erro médio entre a frequência geral e a frequência de caracteres na stream):

$$\chi^2 = \sum_{i=0}^{25} \frac{(O_i - E_i)^2}{E_i}$$

onde O_i é a frequência observada da letra $\text{chr}(\text{ord}('a') + i)$ e E_i é a frequência esperada da

letra que, quando cifrada com deslocamento d, produziria a letra observada. Na prática, o código indexa alfabeto$[(i-d) \mod 26]$ para obter a letra original que teria dado origem à observada.

- Escolhe o deslocamento d que minimiza o valor do qui-quadrado (minimiza o erro).
- Retorna a letra correspondente a esse deslocamento (ex.: deslocamento $4 \rightarrow$ 'd')(lembrando que estamos usando dicionários A1Z26 e não A0Z25). Essa letra é um carácter da chave.

e) Ataque principal – método `decifraMeOuTeDevoro(ciphertext1, ciphertext2)`: **Objetivo:** Para cada uma das duas mensagens (português e inglês), gerar uma lista de chaves candidatas.

Procedimento para cada mensagem:

- Aplica `kasiski()` para obter uma lista de prováveis comprimentos de chave.
- Se a lista estiver vazia (nenhum padrão repetido encontrado), recorre a uma **força bruta** sobre comprimentos de 2 a 10 (tentativa de ataque mesmo com texto curto).
- Limita a lista aos primeiros 5 comprimentos mais prováveis (ou todos, no caso da força bruta).
- Para cada comprimento m na lista:
  - Cria os streams com `criarStreams(ciphertext, m)`.
  - Para cada stream i (0 a $m-1$):
    - Calcula a frequência do stream (`frequencia()`).
    - Chama `mapeamento()` com a língua apropriada ('pt' ou 'en') para obter o caractere mais provável da chave na posição i.
  - Concatena todas as letras obtidas para formar uma chave candidata (tamanho m).
  - Adiciona essa chave à lista de candidatas.
- Retorna uma tupla (candidatas_pt, candidatas_en).

f) Validação e decifragem – método `checagem(resultados, criptogramaPT, criptogramaEN, senha1, senha2)`: **Objetivo:** Verificar se entre as chaves candidatas se encontram as chaves correctas ("senha" e "password") e, em caso afirmativo, decifrar e exibir as mensagens.

Observation 5. Em uma implementação real, o usuário aplicando o código de ataque teria que manualmente percorrer a lista de chaves e aplicá-las 1-a-1 no ciphertext (manualmente ou via código) visando encontrar alguma tradução que produza texto humanamente legível, o que requer análise manual, mas para conveniência do estudo, o método `checagem()` recebe a senha esperada pelo usuário, visando apenas confirmar se o atacante produziu a chave escolhida previamente pelo usuário.

Procedimento:

- Testa se `"senha"` está na lista de candidatas portuguesas e `"password"` na lista inglesa. (ambas senhas escolhidas previamente pelo usuário para o teste do atacante)
- Dependendo do resultado, imprime as mensagens decifradas (usando o método `decifrador()` da própria classe `Atacante`, que implementa a decifragem de Vigenère) ou uma mensagem de falha.

g) Método auxiliar `decifrador(chave, criptograma)`: Implementa a decifragem padrão de Vigenère (similar ao da classe `Vigenere`), preservando os espaços originais, apenas para que o Atacante possa exibir as mensagens decifradas.

C. Análise de Resultados

1) Mensagens curtas (exemplos fornecidos no código):

- **Criptograma PT:** `"uyvkavs pvm g lbtee uhl fspn joe ef tagw"`
- **Criptograma EN:** `"qeosns fi ihw ewb kkpt khasbv lilz dwj kpnvk"`
- As mensagens originais são muito curtas (cerca de 30 caracteres sem espaços).
- O teste de Kasiski **não encontra repetições significativas** (trigramas), retornando lista gerada por análise de bigramas (menos certeira).
- O ataque recorre então ao plano B de utilizar a lista gerada por bigramas.

```
 ATAQUE - MENSAGEM 1 (PT)
Kasiski não encontrou padrões de 3 letras. Tentando com 2...
O Kasiski sugeriu os tamanhos: [2, 7, 14, 11, ...]

 ATAQUE - MENSAGEM 2 (EN)
Kasiski não encontrou padrões de 3 letras. Tentando com 2...
O Kasiski sugeriu os tamanhos: [2, 4, 8, 16, ...]
```

- Devido à reduzida dimensão, a análise de frequência (qui-quadrado) não consegue identificar corretamente os deslocamentos por falta de dados.
- **Resultado final:** `checagem()` indica que o atacante **não foi capaz de encontrar nenhuma das senhas**. O código imprime:
  `"O atacante não foi capaz de encontrar nenhuma das senhas"`

2) Mensagens longas (arquivos `mensagemLongaPT.txt` e `mensagemLongaEN.txt`):

- Os textos originais têm várias centenas de caracteres (sem espaços), o que proporciona repetições e distribuições de frequência robustas.
- O método `kasiski()` detecta várias repetições e sugere comprimentos de chave. Para a mensagem portuguesa, os divisores mais frequentes apontam para o comprimento 5 (a chave `"senha"` tem 5 letras). Para a inglesa, apontam para 8 (`"password"` tem 8 letras).

```
Vamos ver como o atacante se sai:


 ATAQUE - MENSAGEM 1 (PT)
O Kasiski sugeriu os tamanhos: [5, 2, 10, 3, 15, ...]

 ATAQUE - MENSAGEM 2 (EN)
O Kasiski sugeriu os tamanhos: [2, 4, 8, 16, 3, ...]

O atacante encontrou ambas as senhas, aqui estão os textos decifrados:
```

- Para cada comprimento candidato, a análise de frequência com qui-quadrado (comparando com as tabelas de frequência do português e do inglês) consegue determinar corretamente cada letra da chave.
- Entre as chaves candidatas geradas, encontram-se exatamente `"senha"` e `"password"`.
- **Resultado final:** `checagem()` detecta ambas as chaves e decifra as mensagens, exibindo o texto original (que corresponde ao conteúdo dos arquivos). O código imprime:
  `"O atacante encontrou ambas as senhas, aqui estão os textos decifrados:"` seguido dos textos decifrados corretamente.

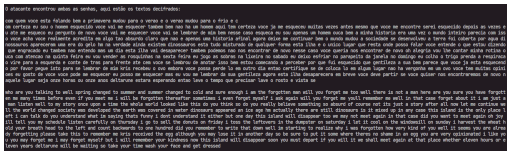

Fig. 1. Textos planos decifrados pelo atacante

IV. Conclusões

Este projeto demonstra a fragilidade da cifra de Vigenère sob análise estatística. Os resultados indicam que a segurança do algoritmo é dependente do volume de dados: mensagens curtas resistem por escassez de amostragem, já textos extensos permitem a descoberta de chaves via Kasiski e Qui-Quadrado. A repetição da chave provou-se uma grande vulnerabilidade, permitindo a decomposição do criptograma em cifras monoalfabéticas triviais. Conclui-se que a implementação valida conceitos de segurança ao mostrar como padrões linguísticos comprometem a segurança de sistemas polialfabéticos.

V. Referências Bibliográficas

References

[1] CIFRA DE VIGENÈRE – Ferramenta online. *Online*. Disponível em: https://cifradevigenere.vercel.app/. Acesso em: abr. 2026.

[2] FREQUÊNCIA DE LETRAS. In: *Wikipédia, a enciclopédia livre*. Disponível em: https://pt.wikipedia.org/wiki/Frequ%C3%AAncia_de_letras. Acesso em: abr. 2026.

[3] VIGENÈRE CIPHER. *Boxentriq*. Disponível em: https://www.boxentriq.com/ciphers/vigenere-cipher. Acesso em: abr. 2026.

[4] MÉTODO KASISKI. In: *Wikipédia, a enciclopédia livre*. Disponível em: https://pt.wikipedia.org/wiki/M%C3%A9todo_Kasiski. Acesso em: abr. 2026.

[5] B. Veitch, "Cryptography - Breaking the Vigenere Cipher," *YouTube*, 4 de mai. de 2021. [Online]. Disponível em: https://www.youtube.com/watch?v=P4z3jAOzT9I. [Acesso em: 5-abr-2026].

VI. Código Fonte

O código fonte para este projeto e implementação dos requisitos solicitados via roteiro de projeto está disponível no GitHub, por meio do link: https://github.com/The3rdMega/Projeto-1---Seguranca---Vigenere